

# Cheat Sheet

## Week 5

### OO programming

- class
    - o class is a type
    - o describes a set of objects that
      - share properties
      - meet requirements
      - adding properties or requirements will make the class more specialized
        - this means objects would need that would be able to match the class would decrease as complexity does
    - o object is an instance of a class
      - example for class Person the object would be = Person alice("Alice", 30);
      - when a program starts, has lot of classes but no instances (most likely)
      - instances created at runtime
      - static objects are exceptions
        - ex. int a = 1; is a static object
  - function overloading vs overriding
    - o overloading
      - basically same function different type
        - void print(int x);
        - void print(double x);
        - void print(const std::string &s)
    - o overriding
      - polymorphism
      - a derived class has it's own version of the base class
        - using virtual in the base class
        - this is considered polymorphism
- ```
class Animal {
public:
    // Mark this function as virtual so it can be overridden
    virtual void speak() const {
        std::cout << "Some generic animal sound...\n";
    }
};

class Dog : public Animal {
public:
    // This overrides the speak() function from Animal
    void speak() const override {
        std::cout << "Woof! Woof!\n";
    }
};
```
- Death
- use destructors
  - only one
  - virtual for polymorphic destruction
- ```
class MyClass {
public:
    ~MyClass() {
        // Cleanup code here
    }
};
```
- ~ means destructor of class
- virtual destructors
    - o virtual ~ base
    - o then you can
    - o ~derived
  - in main function
- ```
int main() {
    Base* ptr = new Derived(); // Allocate a Derived, but store in a Base pointer
    delete ptr;               // Calls both ~Derived() and ~Base() because ~Base() is virtual
    return 0;
}
```

### Lifetime of an object

#### Birth

- executed method when creating object
- method is like a function in a class
- constructor is a member function that is called when a class is created
- default constructor
  - o called when no arguments are provided

```
Student()
: _name("Unknown"), _mt(0.0), _final(0.0)
{
    std::cout << "Default constructor called.\n";
}
```

#### custom constructor

- o called with arguments to setup instance
- ```
Student(const std::string &name, double mt, double fin)
: _name(name), _mt(mt), _final(fin)
{
    std::cout << "Parameterized constructor called.\n";
}
```

#### copy constructor

- o initialized object using another object in class
  - o then this object will copy that object
  - o copy will refer to different place in memory
- ```
Student(const Student &other)
: _name(other._name),
  _mt(other._mt),
  _final(other._final),
  _homeworks(other._homeworks)
{
    std::cout << "Copy constructor called.\n";
}
```

#### move constructor

- o source object passed to constructor
  - o this will then move objects member fields to new object
  - o moves by taking ownership of pointer and that refers to data
  - o then new object now points to original data and source object will no longer point to data
- ```
Student(Student &&other) noexcept
: _name(std::move(other._name)),
  _mt(other._mt),
  _final(other._final),
  _homeworks(std::move(other._homeworks))
{
    // Optionally reset 'other' to safe defaults
    other._mt = 0.0;
    other._final = 0.0;
    std::cout << "Move constructor called.\n";
}
```

#### Protection

- public - available to everyone
- private - class members private are only accessible to member functions inside of the class
- protected - derived class/ subclass can access and inside class
  - o also friend class could help with this

#### Struct Protection vs Class protection

- struct assume pub
  - o can change that by using public: and private:
- class assume private
  - o attributes
  - o methods
  - o chance several times default in calls
    - declaration that follow privacy
    - change follow the new privacy

#### Contract

- two mechanisms
  - o header +implementation files
  - o abstract classes
- contract and implementation is separate

#### Abstract class

- class with state + behavior
- no implementation
- corollary
  - o one never instantiated
  - o one onl sub classes i
- how to use
  - o inheritance to claim contract
  - o provide an implementation in the subclass for pure methods

#### Struct vs. Class

- struct by default is public
- class by default is private
- both have the same structure

#### Styles

- names camel\_case
- Classes and typer Uppercase
- attributes \_underscore
- methods and functions start with lower case

#### OOP

- State + Behavior (have strong cohesion)
  - state - data and object holds
    - o string\_name, double \_mt, \_final, vector <double> \_homeworks
    - o states are protected in classes and are kept private
  - behavior - refers to the operators or functions that act on object data
    - o void read(istream &is)
    - o void print(ostream &os)
    - o behavior acts like a contract to the outside world (public) and its rules are set by class
  - they have strong cohesion because they operate together in one class
- Why do classes have private and public parts
- private
    - o is for data member just for the class
  - public
    - o is for classes functionality for the public to use, these would be like functions

#### Transfer Operators

- simple idea: execute a method invoking assignment
- functions that get called when assign one object to another
- copy operators

```
Student& operator=(const Student &s)
{
    if (this == &s) {
        return *this; // Avoid self-assignment
    }
    // Copy each data member from 's' to 'this'
    _name = s._name;
    _mt = s._mt;
    _final = s._final;
    _homeworks = s._homeworks;

    return *this;
}
```

#### move operators

```
Student& operator=(const Student &s)
{
    if (this == &s) {
        return *this; // Avoid self-assignment
    }
    // Copy each data member from 's' to 'this'
    _name = s._name;
    _mt = s._mt;
    _final = s._final;
    _homeworks = s._homeworks;

    return *this;
}
```

